Lenses & Optics

Season: 7 · Advanced FP · Category: Functional · Level: Advanced

Intent

- Composable immutable updates deep in a structure.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Surgical tweezers for nested records.

When to Use

- Serious FP architecture in Java.
- The design force named in Intent shows up more than once.

When Not to Use

- CRUD apps that don't need effect algebras.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.functional.lens;

import java.util.function.BiFunction;
import java.util.function.Function;

/** Functional getter+setter pair for immutable updates. */
public record Lens<S, A>(Function<S, A> get, BiFunction<S, A, S> set) {
    public S modify(S s, Function<A, A> f) { return set.apply(s, f.apply(get.apply(s))); }

    public <B> Lens<S, B> andThen(Lens<A, B> other) {
        return new Lens<>(
                s -> other.get.apply(get.apply(s)),
                (s, b) -> set.apply(s, other.set.apply(get.apply(s), b))
        );
    }
}

record Address(String city, String zip) {}
record Person(String name, Address address) {}

final class PersonLenses {
    static final Lens<Person, String> name = new Lens<>(Person::name, (p, n) -> new Person(n, p.address()));
    static final Lens<Person, Address> address = new Lens<>(Person::address, (p, a) -> new Person(p.name(), a));
    static final Lens<Address, String> city = new Lens<>(Address::city, (a, c) -> new Address(c, a.zip()));
    static final Lens<Person, String> personCity = address.andThen(city);
}
```

Modern Java 25 / Functional

- Witness types emulate higher-kinded types.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: functional/lens/
```

Pitfalls

- Encodings are educational — production may use libraries.
- Tagless Final needs Kind encoding without HKTs.

Interview Q&A

- Q: When to pick Lenses & Optics? A: When you need testable effect boundaries or pure state threading.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Composable immutable updates deep in a structure.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Surgical tweezers for nested records. [0:40] Intent — State the design force in one sentence: Compos...